

Attention Kernel Portability Across Six GPUs and Two Inference Regimes: A Measurement Study

Srivatsan Sarvesan

University of California, San Diego
ssarvesan@ucsd.edu

Abstract

We study whether an attention backend selected on one GPU remains a good choice on another. We benchmark forward prefill and single-token KV-cache decode on six NVIDIA GPUs with fixed library versions. A backend is the separated winner of a configuration when the runner-up is at least 10% slower and the lower bound of a 95% bootstrap interval on the latency ratio exceeds 1. This holds in 128 of 447 forward-prefill and 409 of 1,203 decode configurations. Among separated comparisons across Ampere, Ada and Hopper GPUs, the winner changes in 27 of 64 prefill comparisons and 29 of 289 decode comparisons. Reusing the source GPU’s choice on the target costs a median $1.002\times$ for prefill and $1.000\times$ for decode, with 95th percentiles of $1.610\times$ and $1.325\times$. In some cases the source’s choice cannot run on the target at all, and we report those separately. Each GPU was measured on its own host, so the results do not separate architecture from host effects.

1 Introduction

A practitioner choosing an attention kernel usually reads a benchmark run on a single GPU. Does the winner of that benchmark stay the winner on a different GPU?

Serving systems usually assume it does. They fix an attention backend by a preference order and use that order on every device they deploy to. If the ranking is a property of the kernel, that is fine. If it is a property of the kernel and device together, an order tuned on one GPU may be wrong on another.

We time 10 prefill and 6 decode implementations over 403 configurations on the six GPUs in Table 1. The 73,230 attempts give 11,835 usable (GPU, configuration, implementation) medians.¹

¹Code: <https://github.com/Srivatsan6923/attention-kernel-portability>. The measurements

Every claim compares a ranking within one device against the ranking within another. We never claim that one device is faster than another.

The study makes two contributions. The first is a measurement protocol that records which kernel actually ran and names a winner only when it is separated from the runner-up. The second is the finding that a backend choice holds only for the GPU, regime and library versions it was measured on.

2 Background and related work

FlashAttention (Dao et al., 2022) keeps the $N \times N$ score matrix out of HBM and is evaluated on A100. FlashAttention-2 (Dao, 2024) is also evaluated on A100, and FlashAttention-3 (Shah et al., 2024) targets H100 only. Each reports results on the GPU its authors measured, which is the practice this study tests. Flash-Decoding (Dao et al., 2023) adds split-KV for single-token queries on A100, and FlashInfer (Ye et al., 2025) builds it into a serving engine evaluated on A100 and H100.

Compiler and kernel benchmarks follow the same pattern. The PyTorch 2 paper (Ansel et al., 2024) evaluates TorchInductor, which generates Triton (Tillet et al., 2019), on one A100. KernelBench (Ouyang et al., 2025) profiles 250 PyTorch workloads on an L40S, and TritonBench (Li et al., 2025) evaluates 184 Triton operators on an A100. Serving systems such as vLLM (Kwon et al., 2023), Sarathi-Serve (Agrawal et al., 2024) and DistServe (Zhong et al., 2024) are evaluated on A100, and Splitwise (Patel et al., 2024) on A100 and H100. TVM (Chen et al., 2018) and Ansor (Zheng et al., 2020) instead search for an implementation per target, which treats the best implementation as a property of the hardware. In HPC the question is well studied. Williams et al.

are in release <https://github.com/Srivatsan6923/attention-kernel-portability/releases/tag/v1.2> with SHA-256 checksums.

(2009) give per-architecture performance ceilings, and Pennycook et al. (2016) and Deakin et al. (2019) define performance portability over a stated set of platforms.

3 Study design

Implementations and grid. The prefill set is a naive PyTorch reference, a `torch.compile` path in three variants, the three PyTorch SDPA backends (memory-efficient, flash and cuDNN), the Triton fused-attention tutorial kernel with a small BF16 change, and FlashAttention-2 and 3. The decode set is a naive KV-cache implementation, an Inductor path, SDPA, `flash_attn_with_kvcache`, FlashInfer, and FlashAttention prefill at query length 1. A *cell* is one configuration, defined by regime, batch, query and KV head counts, head dimension, sequence length, dtype, forward or forward-plus-backward, launch mode, cache state and causality. The full grid has 159 prefill and 244 decode cells. Table 1 gives per-device coverage.

Timing protocol. Backend order is randomised per configuration, and each backend’s timing blocks run back to back. Because backends are not interleaved, we do not assume host or thermal effects cancel within a comparison. In eager mode, each block’s CUDA-event time is divided by its call count. We take the median over blocks within a process launch, then the median over launches. Blocks target about 200 μ s. For cold-cache cells, L2 is flushed once before each block, so only the first call in a block is fully cold. CUDA-graph cells are timed with Triton’s `do_bench_cudagraph`. Confidence intervals come from a 2000-draw bootstrap that resamples whole process launches, because calls within a launch share clock and thermal state. The median between-process coefficient of variation (CV) is 0.011 on A10 (n=1871) and 0.026 on H100 (n=2474).

Processes are pinned to the NUMA node closest to the GPU. In two runs of the H100 decode grid that differ only in pinning, the median between-process CV over 966 matched (cell, implementation) pairs falls from 0.112 unpinned to 0.034 pinned, and the share above 25% CV falls from 30.7% to 9.9%. These numbers use block-level timing and are not directly comparable with the per-call CVs above.

Definitions. Let $t_{g,c,b,r}$ be the median latency of an attention call in process launch r , for GPU g ,

configuration c and backend b . The reported latency is the median over launches,

$$T_{g,c,b} = \text{median}_r t_{g,c,b,r}. \quad (1)$$

With b_1 and b_2 the fastest and second-fastest eligible backends at (g, c) , the separation ratio and rule are

$$R_{g,c} = \frac{T_{g,c,b_2}}{T_{g,c,b_1}}, \quad (2)$$

$$\text{Sep}(g, c) = \mathbf{1}[R_{g,c} \geq 1.10 \wedge L_{95}(R_{g,c}) > 1], \quad (3)$$

where L_{95} is the lower end of a 95% bootstrap interval on the same ratio. Both conditions are required. Failing the rule does not show that the backends are equal. It means no fastest backend could be established. In all 2,082 cells used here, the fastest and second-fastest backends ran in the same set of launches. In the wider scan over every implementation pair, 50 of 29,304 pairs, all on the RTX 5090 and all involving the naive reference, lost a launch to an out-of-memory failure. Those pairs are compared on the launches both backends share.

Let $\mathcal{B}_{g,c}$ be the backends with a usable measurement at (g, c) and $b_{g,c}^*$ the winner. Let $\mathcal{E}$ be the set of matched comparisons (g, h, c) where c was measured on both GPUs and $\text{Sep}(g, c) = \text{Sep}(h, c) = 1$. The winner-change rate is

$$\text{WCR} = \frac{\sum_{(g,h,c) \in \mathcal{E}} \mathbf{1}[b_{g,c}^* \neq b_{h,c}^*]}{|\mathcal{E}|}. \quad (4)$$

The common-backend variant limits each comparison to $\mathcal{B}_{g,c} \cap \mathcal{B}_{h,c}$ for that configuration and recomputes b^* and Sep inside that set.

The cost of reusing GPU g ’s choice on GPU h is

$$C_{g \rightarrow h, c} = \frac{T_{h,c,b_{g,c}^*}}{\min_{b \in \mathcal{B}_{h,c}} T_{h,c,b}}. \quad (5)$$

A value of 1.20 means the source’s choice is 20% slower than the fastest backend on the target. When $b_{g,c}^* \notin \mathcal{B}_{h,c}$ the ratio is undefined. We count those cases and never substitute the target’s own best.

Correctness gate. Every implementation is checked against a reference, and only rows that pass enter a ranking. Of 73,230 attempts, 54,213 (74.0%) completed with status OK. Of those, 54,098 passed the check and are eligible. The other 115 belong to one class that was never checked on its own device and are excluded. Of

GPU	cc	family	mem. BW (GB/s)	planned repeats pre/dec	cells pre/dec
A100	8.0	Ampere	1732.8	5/5	144/192
A10	8.6	Ampere	483.3	5/5	144/191
L40	8.9	Ada	644.4	3/3	144/192
L40S	8.9	Ada	644.4	5/5	144/192
H100	9.0	Hopper	2979.7	5/5	159/244
RTX 5090	12.0	Blackwell	1517.5	5/5	144/192

Table 1: GPUs and coverage. Memory bandwidth is measured on each GPU by timing a 512 MB device-to-device copy and counting bytes read and written. *Planned repeats* are separate process launches per configuration, for prefill and decode. L40 kept a median of two usable repeats. *Cells* counts configurations with at least one eligible measurement.

implementation	completed	unsupp.	OOM	error
<i>prefill</i>				
PyTorch reference	3148	0	851	0
torch.compile	3209	0	790	0
no pattern matching	121	3873	5	0
torch.where mask	121	3873	5	0
SDPA memory-efficient	3969	30	0	0
SDPA FlashAttention	3999	0	0	0
SDPA cuDNN	3999	0	0	0
Triton tutorial kernel	3999	0	0	0
FlashAttention-2	3999	0	0	0
FlashAttention-3	795	3204	0	0
<i>decode</i>				
PyTorch reference	5530	0	10	0
torch.compile	194	5346	0	0
SDPA	5535	0	5	0
FlashAttention-2 KV cache	5515	0	25	0
FlashAttention-2, $q = 1$	5515	0	25	0
FlashInfer	4565	0	25	950

Table 2: Attempt outcomes by implementation, summed over the six GPUs. *Unsupp.* counts configurations the implementation does not support. *OOM* counts attempts skipped by the memory predictor or that ran out of memory. FlashAttention-3 ran only on H100, and all FlashInfer errors are on the RTX 5090.

the remaining attempts, 16,326 were UNSUPPORTED, 1,590 were skipped by an out-of-memory predictor, 950 raised a run-time error (all FlashInfer on the RTX 5090, Section 6), and 151 ran out of memory despite passing the predictor. No correctness check failed. Table 2 breaks these outcomes down by implementation.

4 Dispatch validation

A latency says something about a kernel only if that kernel actually ran. We tried to record the launched kernel names for every timed row and got a readable trace for 36,054 of the 54,213 ok attempts (66.5%). Grouping rows by the fields that decide dispatch (implementation, GPU, head

dimension, dtype, GQA ratio, mode and launch) gives 651 dispatch classes, and 625 of them (96.0%) have at least one traced row. A class confirms the kernel family only. Within a class the exact kernel can change with batch, for example between `flash_fwd_splitkv_combine` and `flash_fwd`, but the family is always the one requested.

Among traced rows, the requested kernel family is missing from the trace in 162 cases, a mismatch rate of 0.449%. Over all ok rows that is 0.299%. We treat the untraced 33.5% as unknown, not as matching. The mismatches cluster, with 66 of the 162 coming from the SDPA memory-efficient backend on H100.

For TorchInductor, 3,645 of 17,537 Inductor rows recorded the `fuse_attention` counter, and it was 0 on every one of them across all six GPUs. Inductor’s attention-fusion pattern therefore never fired on those rows. The counter does not tell the whole story, though. 45 traced Inductor rows on H100, from 6 distinct cells, did launch a flash kernel.

5 Portability

Most configurations have no clear winner. Only 28.6% of forward-prefill and 34.0% of decode configurations have a separated winner (128 of 447 and 409 of 1,203). Figure 1 gives the rate per GPU. A flip between two backends within a few percent of each other is not a measured difference, so Table 3 reports every rate over all shared configurations and over those separated on both GPUs.

Prefill winners change far more often than decode winners. Table 3 pools the ten pairs among A10, A100, L40, L40S and H100. Over all shared configurations the two regimes look similar (66.0% and 52.1%). Keeping only separated comparisons pulls them apart. Prefill changes winner in 27 of 64 comparisons (42.2%) and decode in 29 of 289 (10.0%). Restricting each pair to the backends both GPUs ran, and recomputing winners in that set, gives 33 of 72 for prefill and 27 of 289 for decode. This restriction removes backends that only one GPU could run, such as FlashAttention-3, and it does not lower the prefill rate. Figure 2 shows what reusing a choice costs. The median reused choice is $1.002\times$ the target’s best in prefill and $1.000\times$ in decode, but the 95th percentiles are $1.61\times$ and $1.32\times$.

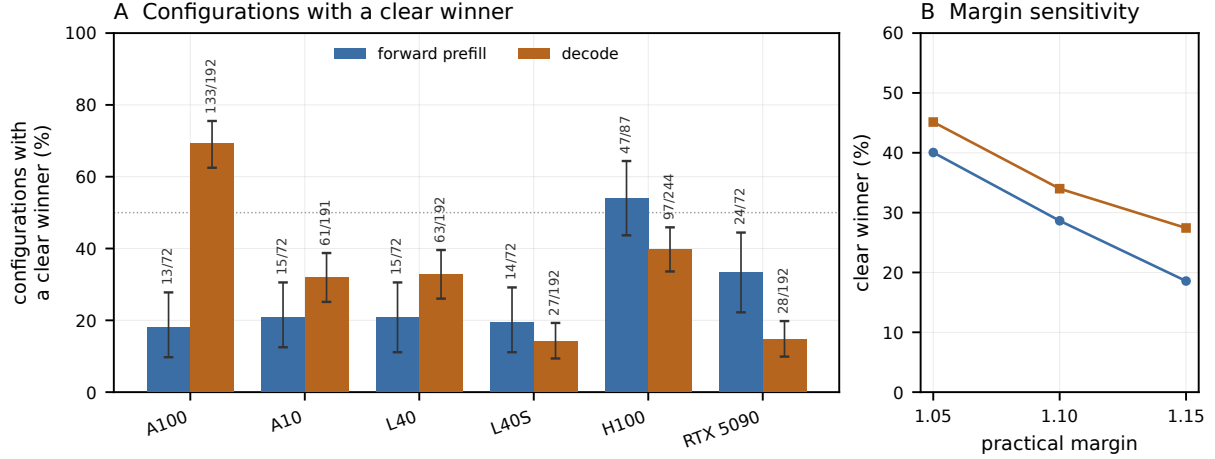


Figure 1: Share of configurations with a clear (separated) winner, by GPU. Whiskers are 95% intervals, and the counts above each bar are separated over total configurations. Workload counts differ by GPU. Panel B shows the pooled rate at margins of 1.05, 1.10 and 1.15.

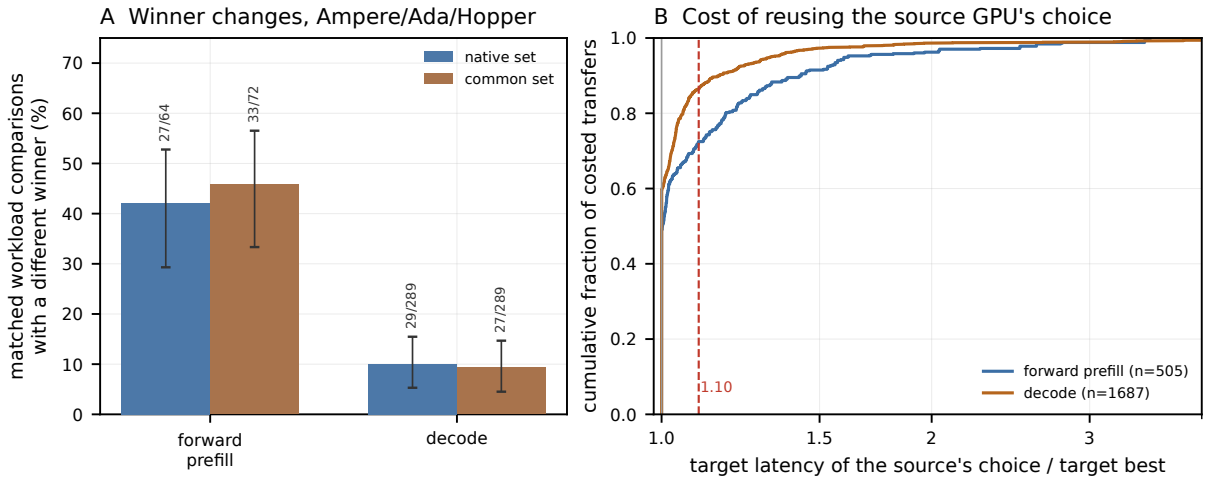


Figure 2: Left, the winner-change rate over the ten Ampere, Ada and Hopper GPU pairs, using each GPU’s own backends (native set) and only the backends both GPUs ran (common set). Right, the distribution of reuse cost over all ordered pairs of the six GPUs where the source has a separated winner. In 85 of 590 prefill and 287 of 1,974 decode cases the source’s choice had no usable timing on the target, and these are not plotted. The cost distribution includes cases where the winner did not change.

The RTX 5090. The lower half of Table 3 repeats the calculation for the five pairs that include the RTX 5090. Decode changes winner in 63 of 85 comparisons and prefill in 16 of 30. In most of the decode cases the other GPU’s winner is FlashInfer, which produced no timing on the 5090 (Section 6). We did not measure what FlashInfer would have cost there, so we do not attribute the high rate to architecture.

6 Backend availability

FlashInfer produced no timing on the RTX 5090. FlashInfer is the decode winner on 128 of 191 cells on A10, 133 of 192 on A100, 131 of 244 on H100 and 87 of 192 on L40, but only

GPU pairs	regime	all shared cells		separated both	
		n	changed	n	changed [95% CI]
Ampere/Ada/Hopper	prefill	720	66.0%	64	42.2% [29, 53]
	decode	1916	52.1%	289	10.0% [5, 15]
RTX 5090 pairs	prefill	360	65.8%	30	53.3% [30, 74]
	decode	959	78.4%	85	74.1% [56, 90]

Table 3: Winner-change rate pooled over GPU pairs. *Ampere/Ada/Hopper* is the ten pairs among A10, A100, L40, L40S and H100, and *RTX 5090 pairs* is the five pairs with the RTX 5090. *n* is the number of pair comparisons and *changed* the share with a different winner. The right columns keep only configurations separated on both GPUs. Intervals are a 2000-draw cluster bootstrap over configurations, since one configuration appears in several pairs.

library	sm_80	sm_86	sm_89	sm_90	sm_120	PTX
flash-attn 2.8.3	SASS	<i>80</i>	<i>80</i>	SASS	SASS	none
libtorch_cuda	SASS	SASS	SASS	SASS	SASS	none
cuDNN engines	SASS	SASS	<i>86</i>	SASS	SASS	sm_120
cuBLAS	SASS	SASS	<i>86</i>	SASS	SASS	sm_120
flashinfer 0.6.18	JIT	JIT	JIT	JIT	none	n/a

Table 4: GPU targets in the installed libraries, from cuobjdump. SASS means native code for that architecture is present. An italic number means no native code shipped and the GPU runs code built for that older architecture, which CUDA allows within one major compute capability. PTX is the embedded PTX the driver can compile for newer GPUs. JIT means the library compiles its kernels at run time, and *none* means no kernel was produced for that target.

49 of 192 on L40S, where SDPA leads. On the RTX 5090 it never ran. Of 960 attempts, 950 raised `RuntimeError: FlashInfer requires GPUs with sm75 or higher across 190 cells`, and the other 10 ran out of memory. The message names sm75 on an sm120 GPU, so it does not explain the failure. Without FlashInfer, the RTX 5090’s decode winner is `flash_attn_with_kvcache` on 137 of 192 cells, at a median of 115.7 μ s.

Library builds. Table 4 lists which GPU targets each installed library was built for. Over forward prefill, FlashAttention-2 wins 35 of 72 cells on A100 and 13 of 72 on the RTX 5090, both of which have native code, but 2 of 72 on L40 and none on A10 or L40S, which run A100 code under binary compatibility. On H100 it wins none of 87, and FlashAttention-3 wins 48. The pattern matches the packaging, but we did not rebuild flash-attn for the missing targets, so we do not claim the packaging causes it. FlashInfer compiles its kernels at run time, so its coverage depends on the host toolkit. Every other library in the audit ships native sm_120 code built with the same CUDA 12.8, so the toolkit can target sm_120 and the FlashInfer failure remains unexplained.

7 Threats to validity

One host per GPU. Each GPU was measured on the host that had it, so device differences are mixed with host differences. A100 prefill was re-measured on a second host because the first run kept too few repeats. Every ranking is formed within one GPU and one regime, so no comparison mixes the two hosts, and their measured memory bandwidths agree within 0.05%. A10 and A100 decode ran a few days before the other runs, on an earlier harness version whose timing code is iden-

tical.

Small separated samples. Requiring a separated winner on both GPUs leaves 64 of 720 pre-fill and 289 of 1,916 decode comparisons among Ampere, Ada and Hopper GPUs. This is the right filter for a ranking claim, but it means the headline rates rest on tens to a few hundred comparisons. One of the 11,835 usable measurements has a single launch and cannot be separated. L40 kept a median of two repeats against five elsewhere, so its intervals are the widest. Dropping it moves the separation rate from 28.6% to 30.1% for pre-fill and from 34.0% to 34.2% for decode.

Untraced rows. 33.5% of OK attempts have no kernel trace, and 26 of 651 dispatch classes have no traced row. The 0.449% mismatch rate applies to traced rows only.

What was not measured. The hosts we used did not allow Nsight Compute or Nsight Systems, so we make no occupancy, cache or counter-based roofline claims. We tested one combination of torch, flash-attn and FlashInfer, and a different version is a different experiment. We time a single attention call, not end-to-end serving latency, on GPUs from one vendor.

8 Conclusion

In the tested configurations, the fastest forward-prefill backend changed across Ampere, Ada and Hopper GPUs much more often than the fastest decode backend. The median cost of reusing a choice was small, but the upper tail and the cases where the choice could not run at all mean the median is not enough to pick a backend. Backend choices should be measured on the GPU and workload they will serve, with uncertainty and coverage reported.

References

- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. [Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve](#). In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable,

- Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, and 30 others. 2024. [PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation](#). In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594.
- Tri Dao. 2024. [FlashAttention-2: Faster attention with better parallelism and work partitioning](#). In *International Conference on Learning Representations (ICLR)*.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. [FlashAttention: Fast and memory-efficient exact attention with IO-awareness](#). In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.
- Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. [Flash-decoding for long-context inference](#). PyTorch Blog.
- Tom Deakin, Simon McIntosh-Smith, James Price, Andrei Poenaru, Patrick Atkinson, Codrin Popa, and Justin Salmon. 2019. [Performance portability across diverse computer architectures](#). In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. [Efficient memory management for large language model serving with PagedAttention](#). In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. [TritonBench: Benchmarking large language model capabilities for generating Triton operators](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23053–23066.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. [KernelBench: Can LLMs write efficient GPU kernels?](#) In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. 2024. [Splitwise: Efficient generative LLM inference using phase splitting](#). In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*.
- S. J. Pennycook, J. D. Sewall, and V. W. Lee. 2016. [A metric for performance portability](#). Preprint, arXiv:1611.07409.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. [FlashAttention-3: Fast and accurate attention with asynchrony and low-precision](#). In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Philippe Tillet, H. T. Kung, and David Cox. 2019. [Triton: An intermediate language and compiler for tiled neural network computations](#). In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19.
- Samuel Williams, Andrew Waterman, and David Patterson. 2009. [Roofline: An insightful visual performance model for multicore architectures](#). *Communications of the ACM*, 52(4):65–76.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasicki, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. [FlashInfer: Efficient and customizable attention engine for LLM inference serving](#). In *Proceedings of Machine Learning and Systems (MLSys)*.
- Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. [DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving](#). In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.